

# RNN 与 GRU 基础笔记

## 从朴素的序列记忆到门控递归建模

2026 年 4 月 6 日

## 目录

|          |                         |          |
|----------|-------------------------|----------|
| <b>1</b> | <b>为什么序列问题需要“记忆”</b>    | <b>2</b> |
| <b>2</b> | <b>贯穿全文的例子</b>          | <b>3</b> |
| <b>3</b> | <b>黑盒视角</b>             | <b>3</b> |
| 3.1      | 输入、处理、输出 . . . . .      | 3        |
| 3.2      | 为什么它不同于普通前馈网络 . . . . . | 3        |
| <b>4</b> | <b>朴素 RNN</b>           | <b>4</b> |
| 4.1      | 基础数学模型 . . . . .        | 4        |
| 4.2      | 一个朴素数值例子 . . . . .      | 5        |
| 4.3      | 算法视角 . . . . .          | 5        |
| <b>5</b> | <b>为什么朴素 RNN 容易吃力</b>   | <b>6</b> |
| 5.1      | “记忆变淡”的直觉 . . . . .     | 6        |
| 5.2      | 实际后果 . . . . .          | 6        |
| <b>6</b> | <b>GRU</b>              | <b>6</b> |
| 6.1      | 核心思想 . . . . .          | 6        |
| 6.2      | 基础数学模型 . . . . .        | 6        |
| 6.3      | 直观算法 . . . . .          | 7        |
| 6.4      | 一个朴素数值例子 . . . . .      | 8        |
| <b>7</b> | <b>RNN 与 GRU 的比较</b>    | <b>8</b> |
| 7.1      | 高层对比 . . . . .          | 8        |
| 7.2      | 权衡与代价 . . . . .         | 9        |

|                          |           |
|--------------------------|-----------|
| 1 为什么序列问题需要“记忆”          | 2         |
| <b>8 神经网络架构本身是怎样学出来的</b> | <b>9</b>  |
| 8.1 从 token 到预测：一条完整管线   | 9         |
| 8.2 各类参数到底在学什么           | 9         |
| 8.3 时间反向传播到底在改什么         | 10        |
| 8.4 为什么 GRU 往往更容易学       | 11        |
| 8.5 这和配套代码为什么能对应起来       | 11        |
| <b>9 关于配套代码的一点说明</b>     | <b>11</b> |
| 9.1 一个真正可以证明的版本          | 12        |
| <b>10 放到统一状态空间视角里看</b>   | <b>14</b> |
| 10.1 为什么这种比较是合理的         | 14        |
| 10.2 两种状态更新到底差在哪里        | 15        |
| 10.3 把这个观点放回贯穿全文的例子      | 15        |
| 10.4 真正值得记住的结论           | 15        |
| <b>11 它们在现实中用来做什么</b>    | <b>16</b> |
| 11.1 典型应用                | 16        |
| 11.2 一个简单工业例子            | 16        |
| <b>12 应当记住什么</b>         | <b>16</b> |

## 1 为什么序列问题需要“记忆”

很多机器学习模型都默认：输入是一组同时给出的数字，模型看一次就可以做判断。比如房价预测，一次性读入面积、楼龄和地段即可；图像分类，也常常只需处理一张静态图片。序列问题不是这样。序列的意义依赖于顺序，而顺序一旦重要，模型就必须具备记忆。

考虑一句很短的话：

```
``the movie is not good''.
```

如果只看最后一个词 `good`，这句话似乎是正面的；但真实含义取决于更早出现的 `not`。因此，实际问题可以用一句很朴素的话概括：

模型怎样才能一边逐步读取序列，一边保留足够的过去信息？

RNN 给出的答案很直接：维护一个隐藏状态，每到一个新时刻就更新一次，把它当作对过去的压缩记忆。GRU 则保留这一主线，但额外加入门控，让模型学会什么该保留，什么该覆盖。

## 2 贯穿全文的例子

全文都围绕一个情感分类器来理解。模型一次读入一个单词，最后判断一条评论是正面还是负面。

我们比较两句话：

```the movie is good''`,      ```the movie is not good''`.

两者的最终任务完全一样：在读完整句话后输出一个标签。真正的难点在于记忆。模型不能只对最后一个词作反应，而要随着句子展开，持续维护一个内部摘要。

为了直观起见，设隐藏状态只是一个实数：

- 正值倾向于表示正面情感，
- 负值倾向于表示负面情感，
- 接近零表示模型还不确定。

这个标量例子远比真实系统简单，但非常适合说明核心逻辑。

## 3 黑盒视角

### 3.1 输入、处理、输出

RNN 和 GRU 都可以看成同一种三段式机器：

1. **输入**。接收序列中的下一项  $x_t$ ；
2. **处理**。将  $x_t$  与上一步传来的记忆状态  $h_{t-1}$  结合起来；
3. **输出**。形成新的记忆状态  $h_t$ ，并在需要时输出  $\hat{y}_t$  或最终预测  $\hat{y}$ 。

二者最大的差别，在于这个新记忆是怎样形成的。朴素 RNN 用一次直接的非线性更新；GRU 则用门控来控制记忆流动。

### 3.2 为什么它不同于普通前馈网络

普通前馈网络通常是“看一眼输入，算完就结束”。而递归模型中的隐藏状态更像一份不断更新的摘要：

旧状态告诉模型它此前知道什么，新输入告诉模型此刻又发生了什么。

这也是它们适合文本、语音、时间序列、轨迹与事件流的原因。

## 4 朴素 RNN

### 4.1 基础数学模型

标准的朴素 RNN 更新公式为

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b),$$

其中

- $x_t$  是时刻  $t$  的输入,
- $h_{t-1}$  是上一步隐藏状态,
- $W_x$  与  $W_h$  是待学习的权重矩阵,
- $b$  是偏置项,
- $\phi$  通常取  $\tanh$  等非线性函数。

若任务是最终分类, 可在最后一步使用

$$\hat{y} = \sigma(W_y h_T + c).$$

这个公式的含义并不复杂。模型在每一步都把两部分信息混合起来:

- 当前看到的新输入;
- 之前保留下来的旧记忆。

输出的  $h_t$ , 就是对前缀  $(x_1, \dots, x_t)$  的新摘要。

把它画成结构图, 会更容易看出信息流向。

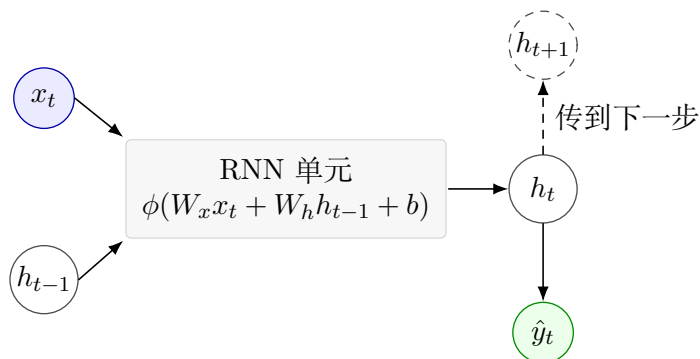


图 1: 朴素 RNN 的一次更新: 当前输入和旧状态先合成新的隐藏状态, 再可选地产生输出。

## 4.2 一个朴素数值例子

设隐藏状态是一维的，并取

$$h_t = \tanh(0.8x_t + 0.5h_{t-1}), \quad h_0 = 0.$$

再给几个词赋予简单的标量输入：

$$x(\text{good}) = 1, \quad x(\text{not}) = -1, \quad x(\text{movie}) = x(\text{is}) = 0.$$

对句子

```movie is good''`,

有

$$h_1 = \tanh(0) = 0, \quad h_2 = \tanh(0) = 0, \quad h_3 = \tanh(0.8) \approx 0.66.$$

最终状态为正，这是合理的。

再看

```movie is not good''`.

此时

$$h_1 = 0, \quad h_2 = 0, \quad h_3 = \tanh(-0.8) \approx -0.66.$$

到最后一个词 `good` 时，

$$h_4 = \tanh(0.8 + 0.5(-0.66)) = \tanh(0.47) \approx 0.44.$$

这就暴露了朴素 RNN 的核心弱点：最后出现的正向词，把状态明显拉回了正值。前面那个 `not` 并没有完全消失，但它的影响已经变弱。若句子再长一些，更早的信息就更容易淡掉。

## 4.3 算法视角

对序列  $x_1, \dots, x_T$ ，朴素 RNN 的算法流程是：

1. 初始化  $h_0$ ；
2. 对  $t = 1, \dots, T$ ，计算  $h_t = \phi(W_x x_t + W_h h_{t-1} + b)$ ；
3. 使用  $h_T$  或全部隐藏状态  $(h_t)$  生成任务输出。

训练时通常使用时间反向传播。做法是先把递归在时间上展开，计算损失，再把梯度沿各个时刻反传回去。

## 5 为什么朴素 RNN 容易吃力

### 5.1 “记忆变淡”的直觉

每次更新，本质上都在把旧状态压缩进新状态。如果这种压缩不断削弱有用信号，那么遥远的信息就会越来越难恢复。可以用一句话概括：

模型像是在不断重写同一本笔记，写得次数多了，旧句子就会越来越模糊。

这就是朴素 RNN 面对长程依赖时常见的问题。它在概念上很优雅，但并不擅长稳固地保存长期记忆。

### 5.2 实际后果

因此，朴素 RNN 更适合如下场景：

- 序列较短；
- 关键信息主要来自局部上下文；
- 计算预算较紧，希望模型尽量简单。

若任务要求跨越许多步仍保存关键线索，它就往往不够稳健。

## 6 GRU

### 6.1 核心思想

GRU 的全称是 Gated Recurrent Unit。它保留了递归结构，但在内部加入了门控。所谓门控，本质上是介于 0 到 1 之间的学习型系数，用来决定新信息应进入多少、旧信息应保留多少。

因此，GRU 真正回答的问题不是“怎样生成一个隐藏状态？”而是“怎样在更新记忆时，不把有用历史过快抹掉？”

### 6.2 基础数学模型

常见的 GRU 形式写作

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z),$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r),$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h),$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

其中

- $z_t$  是更新门,
- $r_t$  是重置门,
- $\tilde{h}_t$  是候选状态,
- $\odot$  表示逐元素乘法。

虽然公式比朴素 RNN 长, 但直觉其实很简单:

- 更新门决定旧记忆应保留多少、新记忆应写入多少;
- 重置门决定在构造候选记忆时, 要参考多少旧状态。

把门控结构画出来, 可以把这两个作用分开看。

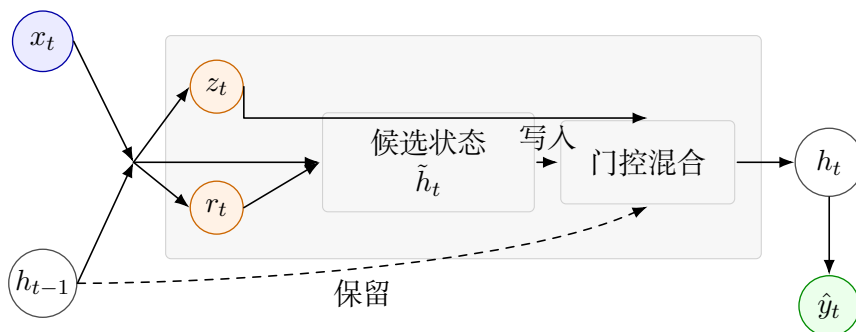


图 2: GRU 的门控更新: 更新门控制写入比例, 重置门控制候选状态对旧信息的依赖。

### 6.3 直观算法

在每个时刻, GRU 的行为可以理解为:

1. 读入新输入  $x_t$ ;
2. 先构造一个候选新记忆  $\tilde{h}_t$ ;
3. 再判断旧记忆要保留多少;
4. 把旧记忆与候选记忆按比例混合, 得到  $h_t$ 。

关键改进就在于这个“混合”。朴素 RNN 更像一次性重写记忆; GRU 则更像受控覆盖。

## 6.4 一个朴素数值例子

回到句子

``movie is not good''.

设隐藏状态仍是一维，并假设网络已经学会如下行为：

- 读到 `not` 后，会存下一个较强的负向记忆；
- 遇到语义较弱的填充词时，会尽量保留这段记忆；
- 遇到 `good` 这类情感词时，会把新证据与旧上下文混合，而不是完全覆盖。

设读到 `not` 之后有

$$h_2 = 0, \quad h_3 = -0.80.$$

若下一个词只是弱填充词，且 GRU 选择

$$z_4 = 0.10, \quad \tilde{h}_4 = -0.40,$$

则

$$h_4 = (1 - 0.10)(-0.80) + 0.10(-0.40) = -0.76.$$

可见负向记忆被大体保住了。

最后遇到 `good` 时，若模型计算得到

$$z_5 = 0.60, \quad \tilde{h}_5 = 0.20,$$

则

$$h_5 = 0.40(-0.76) + 0.60(0.20) = -0.184.$$

最终状态仍略偏负，这比前面朴素 RNN 得到的正值结果更符合语义：`good` 的确重要，但它不应彻底抹去先前的 `not`。

## 7 RNN 与 GRU 的比较

### 7.1 高层对比

| 方面      | 朴素 RNN    | GRU          |
|---------|-----------|--------------|
| 记忆更新方式  | 一次非线性直接改写 | 旧记忆与新记忆的门控混合 |
| 参数规模    | 较小        | 较大           |
| 训练稳定性   | 长序列上常较弱   | 通常更好         |
| 保留上下文能力 | 有限        | 更强           |
| 理解门槛    | 更适合作为入门模型 | 更接近实用基线      |



## 7.2 权衡与代价

朴素 RNN 的价值，在于它是最干净的递归思想。它容易教学、容易实现、参数也少。但这种简洁，是以记忆脆弱为代价的。

GRU 则通常是更实用的选择，尤其当任务需要跨越多步保留上下文时更是如此。代价是结构更复杂、参数更多。更准确的理解方式不是“GRU 推翻了 RNN”，而是“GRU 修补了 RNN 最主要的一类工程弱点”。

# 8 神经网络架构本身是怎样学出来的

## 8.1 从 token 到预测：一条完整管线

理解这件事时，最好把架构和优化器分开。优化器也许是 Adam，也许是普通梯度下降，但它优化的对象始终是一条具体的神经网络管线：

1. 先把 token 或输入值映射成向量表示；
2. 再让递归核心把这个新向量与旧隐藏状态结合起来；
3. 再由读出层把最终隐藏状态变成任务输出；
4. 最后把预测误差变成梯度，沿整条链路反传回去。

因此，模型学到的并不是一个笼统的“黑箱能力”，而是几类参数一起协调出来的结果。嵌入层学习怎样表示词；递归核心学习怎样更新记忆；读出层学习怎样把隐藏状态解释成最终预测。

## 8.2 各类参数到底在学什么

**嵌入层。** 嵌入矩阵学习的是：哪些 token 应该在向量空间里彼此接近，哪些应当被分开。在情感任务中，模型也许会学到 good 与 great 的方向比较接近；而 not 则更像一个修饰器，它不直接表达情感，却会改变后面词的作用方式。

**递归核心。** 递归权重决定新输入和旧状态如何融合。在朴素 RNN 里，同一套更新机制必须同时完成许多任务：

- 对新信息作出反应；
- 保留有用旧信息；
- 丢掉无关细节；
- 还要让整个动力学保持数值稳定。

这正是朴素 RNN 容易难训的原因之一。

而在 GRU 里，架构本身已经把这些职责拆得更开。更新门学习什么时候主要保留旧记忆；重置门学习什么时候在构造候选状态时应弱化旧状态。换句话说，GRU 不是凭空“更聪明”，而是它先给优化器提供了一个更有结构的参数化方式。

**读出层。** 读出层学习的是如何把隐藏状态空间中的几何方向翻译成任务标签。对二分类情感任务而言，可以把它理解成：读出层在隐藏状态空间里学出一条“更正面”与“更负面”的方向。隐藏状态本身不一定天然可解释，只要经过读出后能服务预测即可。

### 8.3 时间反向传播到底在改什么

时间反向传播这个词常常听起来很抽象，所以不妨用句子

```
``movie is not good''
```

来把它拆开。

假设模型错误地把这句话判成了正面。

**enter: 先定位错误。** 最终输出之所以错，是因为最后的隐藏状态被读出层解释得过于正面。

**rw: 先把误差送回读出层。** 读出层权重会收到梯度，告诉它当前这个最终状态不该再如此强地支持正类。

**rw: 再把误差送回递归核心。** 递归参数会收到另一个信号：not 的影响没有被足够稳定地保留到最后，因此后面的 good 才会被解释得过于字面。

**rw: 继续送回嵌入层。** not 这个 token 的嵌入也会收到学习信号。模型实际上被告知：

这个词应当把隐藏状态推向一个能持续更久、足以改变后续词语解释的方向。

**done: 所有参数组一起更新。** 一次优化步之后，模型并不是简单地“背下一个标签”，而是同时微调了：

- not 的向量表示；
- 记忆如何向后传递的递归规则；
- 最终隐藏状态如何被解释的读出规则。

这就是为什么架构重要。不同的参数化方式，会决定同一个误差信号究竟能沿哪些路径传播。

## 8.4 为什么 GRU 往往更容易学

GRU 的优势并不来自什么神秘力量。更准确的说法是：它往往更容易训练，因为它给优化器提供了一个更友好的搜索空间。

在朴素 RNN 里，一条递归映射必须同时决定：

- 写入多少新信息；
- 保留多少旧信息；
- 梯度应当如何跨时间存活。

在 GRU 里，这些职责被拆得更清楚。这样并不保证它在每个小任务上都赢，但它确实减少了“优化器和架构彼此对抗”的情况。因此，只要任务真的开始依赖长记忆，GRU 往往更容易被训练到正确的工作状态。

## 8.5 这和配套代码为什么能对应起来

从这个角度看，easy toy demo 和 harder benchmark 的差异也就更容易理解了。

在 easy sentiment demo 里，架构不需要学会非常精细的记忆策略。句子短、模式简单，因此 plain RNN 和 GRU 都可能找到够用的参数组合。

但在 long-memory benchmark 里，架构必须学会更苛刻的行为：

尽早写入一位信息，把它穿过很多干扰步骤保护住，再在最后把它读出来。

这正是 GRU 架构内部职责分离开始真正发挥价值的地方。因此，这个 benchmark 讲的不只是精度差异，也是在讲：哪一种神经网络架构更容易被训练成所需的记忆行为。

# 9 关于配套代码的一点说明

这份笔记配套的小型情感分类 demo 是故意做得很容易的：数据集很小，句子很短，依赖关系也很简单。在这种条件下，朴素 RNN 和 GRU 看起来往往会比较相近，因为二者都还没有被真正逼到“必须处理很长记忆”的地步。

因此，这个 toy demo 更适合被理解为**机制演示**，而不是决定性的 benchmark。它帮助读者看到递归状态如何一步一步被更新，但它本身并不能单独证明 GRU 在工程上一定有明显优势。

如果想更诚实地看出二者的工程差别，就需要更难的 benchmark：更长序列、更远距离的依赖，或者训练长度和测试长度之间的外推。在这种场景下，GRU 往往会更稳定、更可靠。但更准确的说法仍然应当是：

GRU 在更难的记忆问题上通常更容易训练，并不意味着它在所有小任务上都必然优于朴素 RNN。

现在这套配套代码里已经加入了这样一个更难的 benchmark, 即 `code/long_memory_benchmark.py`。它故意把训练和测试做成不对称: 模型训练时只看到长度为 12 的短序列, 但测试时要面对最长到 96 的更长序列。任务本身也更贴近“真正考记忆”: 模型需要记住序列开头写入的一位信息, 并在经历很长一串干扰 token 之后, 到了末尾再把它读出来。

在一次代表性实跑里, 朴素 RNN 在这些测试长度上的平均准确率大约是 0.502, 而 GRU 大约是 1.000。这个数字的意义并不是“GRU 从此普遍胜出”, 而是一个更克制也更有用的结论: 当任务真的开始考验长程记忆和长度外推时, 直接反复改写状态与带门控的记忆更新之间的工程差异, 就会更容易被观察到。

| 模型     | 训练长度 | 跨测试长度平均准确率 |
|--------|------|------------|
| 朴素 RNN | 仅 12 | 约 0.502    |
| GRU    | 仅 12 | 约 1.000    |

## 9.1 一个真正可以证明的版本

句子“GRU 收敛, 而朴素 RNN 不收敛”本身太强, 不能直接当成定理来用。一般地说, 两类模型都可能成功, 也都可能失败, 这取决于参数、初始化、优化过程以及任务本身。

对于上面的 long-memory benchmark, 更可辩护的严格说法其实是下面这对命题:

1. 一类收缩型朴素 RNN 会指数遗忘早期信息;
2. GRU 存在一组参数, 使得在任意固定时域内, 记忆误差都可以做得任意小。

**命题 1 (收缩型朴素 RNN 指数遗忘)。** 考虑标量递推

$$h_t = \tanh(\alpha x_t + \beta h_{t-1}), \quad |\beta| \leq \rho < 1.$$

取两条序列, 它们从时间 2 开始完全相同, 只在时间 1 写入了不同的 bit。对应隐藏状态分别记为  $h_t^+$  与  $h_t^-$ 。则有

$$|h_T^+ - h_T^-| \leq \rho^{T-1} |h_1^+ - h_1^-|.$$

**按小步证明。** **enter:** 先定义差值

$$\Delta_t := h_t^+ - h_t^-.$$

**rw:** 因为两条序列在  $t \geq 2$  时使用的是同一个输入  $x_t$ , 所以

$$\Delta_t = \tanh(\alpha x_t + \beta h_{t-1}^+) - \tanh(\alpha x_t + \beta h_{t-1}^-).$$

**have:** 函数  $\tanh$  是 1-Lipschitz 的, 因此

$$|\tanh(u) - \tanh(v)| \leq |u - v|.$$

在这里代入

$$u = \alpha x_t + \beta h_{t-1}^+, \quad v = \alpha x_t + \beta h_{t-1}^-,$$

便得到

$$|\Delta_t| \leq |\beta| |h_{t-1}^+ - h_{t-1}^-| = |\beta| |\Delta_{t-1}| \leq \rho |\Delta_{t-1}|.$$

**done:** 从  $t = 2$  一直迭代到  $t = T$ , 便有

$$|\Delta_T| \leq \rho^{T-1} |\Delta_1|.$$

于是, 两种不同早期 bit 所对应的分离边距会随着序列长度指数衰减。

**直观解释。** 这并不是在说所有朴素 RNN 都失败。它说的是: 一旦递推映射本身是收缩的, 那么早期信息就会被指数级压扁。这正是长程记忆和长度外推容易变脆弱的核心机制。

**命题 2 (GRU 可实现任意精度的有限时域记忆)。** 考虑标量 GRU 状态

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t, \quad 0 \leq z_t \leq 1.$$

固定时域  $T$  与目标精度  $\delta > 0$ 。设要保存的 bit 对应目标值  $s \in [-m, m]$ 。若参数被选到满足

$$|h_1 - s| \leq 2m\varepsilon, \quad z_t \leq \varepsilon \text{ 对所有 } 2 \leq t \leq T, \quad |\tilde{h}_t| \leq m \text{ 对所有 } 2 \leq t \leq T,$$

那么对任意  $1 \leq t \leq T$ , 都有

$$|h_t - s| \leq 2mt\varepsilon.$$

因此, 只要取

$$\varepsilon \leq \frac{\delta}{2mT},$$

就能保证

$$|h_t - s| \leq \delta \quad \text{对所有 } 1 \leq t \leq T.$$

**按小步证明。** **enter:** 定义记忆误差

$$e_t := |h_t - s|.$$

**rw:** 将 GRU 更新式代入:

$$e_t = \left| (1 - z_t)h_{t-1} + z_t \tilde{h}_t - s \right|.$$

在右边加减  $(1 - z_t)s$ , 可改写成

$$e_t = \left| (1 - z_t)(h_{t-1} - s) + z_t(\tilde{h}_t - s) \right|.$$

**have:** 用三角不等式:

$$e_t \leq (1 - z_t)|h_{t-1} - s| + z_t|\tilde{h}_t - s| = (1 - z_t)e_{t-1} + z_t|\tilde{h}_t - s|.$$

**rw:** 因为  $|\tilde{h}_t| \leq m$  且  $|s| \leq m$ , 所以

$$|\tilde{h}_t - s| \leq |\tilde{h}_t| + |s| \leq 2m.$$

再结合  $z_t \leq \varepsilon$ , 得到

$$e_t \leq e_{t-1} + 2m\varepsilon.$$

**done:** 从  $e_1 \leq 2m\varepsilon$  出发迭代:

$$e_t \leq e_1 + 2m(t-1)\varepsilon \leq 2mt\varepsilon.$$

于是只要令  $\varepsilon \leq \delta/(2mT)$ , 便可保证整个时域内  $e_t \leq \delta$ 。

**为什么这和 benchmark 对得上。** 这个 benchmark 的 token 类型是有限个, 所以 GRU gate 只需要在有限种输入情形下“表现正确”即可。对于有限集合, sigmoid 的 logit 完全可以在干扰 token 上做得非常负, 在写入 token 上做得非常正。因此, 上面的假设  $z_t \leq \varepsilon$  并不是空想; 它正是 GRU 在这类任务里能够学出来的门控模式。

**真正应记住的结论。** 因此, 严格说法不应是 “GRU 收敛, 而 RNN 不收敛。” 更准确的说法是:

对这类 long-memory 任务, 一类收缩型朴素 RNN 会指数遗忘早期信息, 而 GRU 则允许在任意固定时域上实现近似恒等的记忆传递。

这个结论在数学上是站得住的, 也更贴近 benchmark 真正展示的内容。

## 10 放到统一状态空间视角里看

### 10.1 为什么这种比较是合理的

如果站得足够高, GRU 与 Kalman filter 的确有家族相似性。它们都按时间一步一步处理数据流; 都维护一个内部状态; 都在当前时刻利用新到来的信息去更新这个状态。从这个抽象层面看, 它们都可以放进同一个状态更新框架:

新状态 = 更新规则(旧状态, 当前输入).

这才是更准确的统一说法。相较之下, “它们都在预测未来动作” 往往并不严谨。很多任务里, 它们根本不直接预测动作; 它们真正做的, 是持续维护一个对历史信息的内部摘要。

## 10.2 两种状态更新到底差在哪里

对 Kalman filter 而言，内部状态是对外部真实状态的估计，并且通常伴随一份不确定性描述。在控制场景中，常见写法是

$$\hat{x}_{t|t-1} = f(\hat{x}_{t-1|t-1}, u_{t-1}), \quad \hat{x}_{t|t} = \text{校正}(\hat{x}_{t|t-1}, y_t),$$

其中  $u_{t-1}$  是控制输入， $y_t$  是新的观测。它的更新是**模型驱动**的：工程师先写出动力学和观测关系，滤波器再据此估计潜在状态。

对 GRU 而言，内部状态是一个学习得到的隐藏表示。其更新可概括为

$$h_t = \text{GRUCell}(h_{t-1}, x_t),$$

其中  $x_t$  是下一个词、下一个传感器值，或者下一个时间序列样本。它的更新是**数据驱动**的：不是先由物理模型手工指定，而是从样本中学出来。

因此，二者的结构相似性是真实存在的：

- 都是在递归地更新状态；
- 都在把过去压缩成当前状态；
- 都会用当前输入去修正这个状态。

但它们的工作方式同样有本质差别：

- Kalman filter 是带显式不确定性账本的状态估计器；
- GRU 是一个学习型记忆机器，其隐藏状态通常没有直接的物理语义。

## 10.3 把这个观点放回贯穿全文的例子

在情感分析例子中，GRU 的隐藏状态保存的是一句话前缀的摘要。读到 `not` 之后，状态应把这个负向修饰继续带下去，从而让后面的 `good` 被放到正确上下文中理解。

在 Kalman filter 的典型例子里，我们可能在跟踪机器人位置和速度。滤波器会把当前最优估计先向前推进，再根据新的含噪传感器读数进行修正。两者在“记忆”的角色上因此是相似的：当前状态都在替过去的有效信息做摘要，等待下一条数据到来。

但这个状态的含义不同。情感分析里的隐藏状态，是为了预测任务而学到的有用内部表示；机器人跟踪里的 Kalman 状态，则试图对应一个客观存在但无法被完美观测的外部量。

## 10.4 真正值得记住的结论

如果要用一句统一的话来概括，可以说：

Kalman filtering 和 GRU 都属于更广义的序贯状态更新方法，但前者是带不确定性的模型驱动估计器，后者是数据驱动的隐藏状态更新器。

这个视角的价值在于，它既避免把两者说成完全一样，也避免把两者看成毫无关系。更高层的共同骨架其实是：

1. 先维护一个状态；
2. 新数据到来时更新它；
3. 再把更新后的状态用于估计、预测或决策。

## 11 它们在现实中用来做什么

### 11.1 典型应用

凡是数据按顺序到来的地方，都可能出现这两类模型：

- 语言建模与情感分析，
- 语音与手写识别，
- 传感器预测与异常检测，
- 医疗时间序列，
- 用户行为建模。

### 11.2 一个简单工业例子

设工厂里有一台机器，温度序列为

$$x_1, x_2, \dots, x_T.$$

任务是预测它在下一分钟内是否会过热。

若模型只看当前温度，可能会漏掉持续上升的趋势；而递归模型能够记住温度在很多步上的变化方向。尤其是 GRU，当中间偶尔出现一次短暂降温时，它也不必立刻忘记之前已经积累的整体上升趋势。

## 12 应当记住什么

这条概念链条其实很短：

1. 序列任务要求模型具备记忆；
2. 朴素 RNN 通过反复更新隐藏状态来形成记忆；
3. GRU 保留递归主线，但用门控让记忆更新变得有选择。



若必须用一句话概括二者差别，可以说：

朴素 RNN 每一步都在重写记忆，而 GRU 学习何时保留、何时遗忘、何时刷新。

因此，RNN 更适合用来理解序列建模的原始思想；GRU 则往往是更稳妥的工程选择。